
EECS 348: Software Engineering I

EECS 348 Group 5

User's Manual

Version <1.6>

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

Revision History

Date	Version	Description	Author
05/04/2026	1.1	FAQ Section	Lea
05/05/2026	1.2	Introduction Section	Alex
05/05/2026	1.3	Getting Started Section	Shashwat
05/05/2026	1.4	Advanced Features Section	Zack
05/05/2026	1.5	Troubleshooting Section	Jr
05/06/2026	1.6	Purpose Section and revision of document	Alec

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

Table of Contents

1. Purpose	4
2. Introduction	4
3. Getting started	4
Running the Program	4
Supported Operations	4
Example Inputs	5
Expression Parsing Structure	5
Error Handling	5
4. Advanced features	5
5. Troubleshooting	6
6. Examples	6
7. Glossary of terms	7
8. FAQ	8

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

User Manual

1. Purpose

This user manual provides a guide for using the Arithmetic Expression Evaluator. It explains how to compile and run the program from the command line, how to enter and evaluate arithmetic expressions, and how to interpret both successful results and error messages. The manual also includes practical examples and a troubleshooting section to help users understand and resolve common issues.

2. Introduction

The Arithmetic Expression Evaluator is a program that allows users to perform complex computations instantaneously. More precisely, it identifies numerals and operators within a text string, translates them into a mathematical structure, and evaluates the input into a final value following the standard order of operations (PEMDAS). To compile and run the program, ensure a C++ compiler is installed and enter `g++ parser.cpp -o evaluator` in your terminal, then launch the application with the command `./evaluator`.

3. Getting started

The Arithmetic Expression Evaluator is a command-line program that evaluates mathematical expressions using a recursive descent parser. The software supports arithmetic operations, parentheses, unary operators, and exponentiation while following standard operator precedence rules.

Running the Program

After compiling and running the program, the user will be prompted to enter an expression. The statement below will be shown.

“Enter your expression:”

Supported Operations

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo
**	Exponentiation

The program also supports parentheses, decimal numbers, and unary operators such as -5 and +3.

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

Example Inputs

2 + 3

10 * (4 + 2)

2 ** 3

-(5 + 1)

5 % 2

3.5 + 2.1

Expression Parsing Structure

The parser evaluates expressions using the following hierarchy:

Expression → + -

Term → * / %

Power → **

Unary → + -

Primary → numbers and parentheses

This structure ensures that higher-precedence operations are evaluated before lower-precedence operations.

Error Handling

The program detects invalid expressions and displays appropriate error messages.

- “Division by zero”
- “Missing closing parenthesis”
- “Invalid number format”
- “Unexpected characters in the expression”

4. Advanced features

The parser includes support for **floating-point arithmetic**, allowing users to input decimal values and obtain precise results. Outputs are formatted dynamically: decimal values are displayed only when necessary, while whole-number results are presented without unnecessary trailing zeros for improved readability.

The system also provides robust handling of **parenthetical expressions**. During evaluation, the primary parsing stage verifies that every opening parenthesis has a corresponding closing parenthesis. If a mismatch is detected, the parser will terminate execution and return an appropriate error message, ensuring input validity.

Additionally, the parser supports **exponentiation** using the ****** operator. A dedicated parsing function checks specifically for two consecutive asterisk (*) characters to distinguish exponentiation from multiplication. If only a single * is detected, the parser safely reverts to its previous position to correctly interpret the operation as multiplication, preventing unintended character skipping and preserving parsing accuracy.

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

5. Troubleshooting

- **Compiling Error:** If the user faces errors when trying to run the program on a local device, they must ensure that they have the GCC compiler installed on their device. Instructions for GCC installation can be found [here](#).
- **Unmatched Parentheses:** The user may encounter an error message-"Invalid expression, unclosed parentheses." If this occurs, the user should re-evaluate their expression and ensure there is an equal number of open and closed parentheses. E.g., "Missing closing parenthesis."
- **Unaccepted Operators:** The parser accepts a previously determined set of arithmetic operators, as stated throughout multiple documents. The use of the common, non-accepted operators such as $^{\wedge}$ to represent exponentiation will not be recognized by the parser and will be met with an error message. Using ** to notify of exponentiation suffices.
- **Division/Modulo by zero (0):** Division and modulo by 0 result in impossible calculations for the parser. In the case that the user is met with this error, they should ensure that their expression does not simplify into an expression that results in division or modulo by 0. E.g., "Division by zero"/"Modulo by zero".
- **Incomplete decimal values:** If the parser encounters an incomplete/unfinished decimal value (5.), the user will receive an error message. To correct this behavior, ensure that the entered expression does not include any incomplete decimals. E.g., "Invalid number format".
- **Lagging Characters:** In the case that there are leftover characters after evaluating a valid expression, the parser will throw an error. For example, if the parser is given the following expression, it will throw an error: $3 + 4)$. Ensure that the expression does not have any invalid lagging characters. E.g., "Invalid expression: unexpected characters at the end."
- **Invalid Input:** The parser does not accept non-numerical characters outside of arithmetic operators. This being said, if an alphabetical letter is entered into the parser, the user will be met with the following error: "Invalid expression." Ensure that no alphabetical characters are entered into the user's expression when faced with this error.

6. Examples

Example 1: Basic Operations

- Input entered by user: $10 * 2 / 5$
- Program Output: 4
- Explanation: The multiplication and division operators are applied from left to right.

Example 2: Parentheses and Precedence

- Input entered by user: $8 - (5 - 2)$
- Program Output: 5
- Explanation: Parentheses are evaluated first, forcing the inner subtraction to occur before the outer operation.

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

Example 3: Exponentiation

- Input entered by user: $2 ** 3$
- Program Output: 8
- Explanation: The $**$ operator performs exponentiation. It has a higher precedence than other operators and is evaluated right-to-left

Example 4: Unary Operators

- Input entered by user: $- (-3) + 2$
- Program Output: 5
- Explanation: The program correctly handles unary negation, including when it is nested inside parentheses.

Example 5: Mixed Operators with Parentheses

- Input entered by user: $4 * (3 + 2) \% 7 - 1$
- Program Output: 5
- Explanation: The expression inside the parentheses is evaluated first, followed by multiplication, modulo, and subtraction from left-to-right.

7. Glossary of terms

PEMDAS (Order of Operations):

The hierarchical rule set used by the evaluator to resolve expressions. The program processes the input string in the following priority:

1. Parentheses (): Expressions inside brackets are isolated and solved first.
2. Exponents : Power calculations are performed next.
3. Multiplication $*$ and Division $/$ and Modulo $\%$: These are handled from left to right.
4. Addition $+$ and Subtraction $-$: These are the final operations performed.

Addition (+):

A binary operator that sums the numerical values on its left and right. It is evaluated with the lowest precedence.

Subtraction (-):

A binary operator that calculates the difference between the left and right values. It can also function as a Unary Operator to indicate a negative number.

Multiplication (*):

A binary operator used to scale the first value by the second. It holds higher precedence than addition and subtraction.

Division (/):

A binary operator that computes the quotient of two values.

Modulo (%):

A binary operator that returns the remainder of a division operation.

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

Exponentiation ():**

A multi-character binary operator used to raise a base to a power. It is the only operator in this evaluator that is right-associative.

Binary Operator:

An operator that requires two numbers to perform a calculation.

Unary Operator:

An operator that requires only one number to function. In this program, it is used to indicate whether a value is positive or negative.

Floating-point Arithmetic:

A method used by the evaluator to process and calculate numbers that include decimal points.

Parenthetical Expression:

A part of a mathematical formula enclosed within parentheses.

Compilation Error:

An error that occurs when the compiler cannot translate source code into an executable program.

Troubleshooting:

The systematic process of identifying, isolating, and resolving faults or "bugs" within a system.

Character:

A single unit of information that corresponds to a symbol.

8. FAQ

Basic Functionality

- I. What mathematical operations does this calculator support?**
 - A. This calculator supports addition (+), subtraction (-), multiplication (*), division (/), modulo/remainder (%), and exponentiation/power (**).
- II. Does the calculator support decimal numbers?**
 - A. Yes, you can input both whole numbers and decimal numbers. For example, $3.14 + 2.6$ will give you the result 5.74 .
- III. How do I calculate exponents/powers?**
 - A. To raise a number to a power, use double asterisks (**). For example, to calculate 3 squared, you would type $3 ** 2$.
 - B. NOTE: Don't use the caret symbol (^); it doesn't support exponents in this parser.
- IV. Does the calculator follow the standard order of operations (PEMDAS/BODMAS)?**
 - A. Yes, the parser automatically respects mathematical precedence:
 1. Parentheses ()
 2. Exponents **
 3. Multiplication *, Division /, and Modulo %
 4. Addition + and Subtraction -

<EECS 348 Group 5>	Version: <1.0>
User's Manual	Date: <05/07/2026>

V. Can I use spaces between numbers and operators?

- A. Yes, you can format your expression however you like. The parser automatically ignores whitespace. For example, $6+4$ and $6 + 4$ and $6 + 4$ will all yield the same result of 10 .

VI. Can I use negative numbers?

- A. Yes, the parser supports unary operators, meaning you can place a minus sign (-) or plus sign (+) directly in front of a number or parenthesis to denote a positive or negative value. For example, you can type $-7 + 3$ or $-(8 * 4)$, and the calculator will handle them perfectly.

Troubleshooting & Errors

I. Why did I get a “Division by zero” error?

- A. Usually in math, dividing by zero is undefined. If you attempt to solve an expression like $10 / 0$, the calculator automatically intercepts it and provides this error to prevent the program from crashing.

II. Why did I get a “Modulo by zero” error?

- A. Similar to division by zero, you can't find the remainder of a number divided by zero. So, entering an expression like $2 \% 0$ will trigger this error.

III. What causes the “Missing closing parenthesis” error?

- A. This happens when you open a parenthesis but forget to close it. For example, if you typed in $(9 + 3 * 1$, the expression is incomplete. You must add the matching $)$ to create a valid expression.

IV. What does the “Invalid number format” error mean?

- A. This error occurs when you type a number that the parser cannot understand, this might be because of too many decimal points. For example, if you type $1.2.4$ it will trigger this error.

V. Why did I get the error: “Invalid expression: unexpected characters at the end”?

- A. This happens when the parser successfully reads a valid expression, but then finds extra characters that don't make sense. For example, when you type $1 + 4)$ or $5 * 3 abc$, this will trigger the error.

VI. My expression is very long. Is there a size limit?

- A. For normal calculations, there is no limit. The calculator can easily process massive equations. You would only run into an error if you tried to enter an impossibly long problem that runs your computer out of memory.